

IF3270 Pembelajaran Mesin
Tugas Besar 2



Disusun oleh Kelompok Amin (55):

Muhammad Adam M. (18223015)

Devon Wiraditya T. (18223039)

Dosen Pengampu:

Dr. Fariska Zakhralativa Ruskanda, S.T., M.T.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
JL. GANESHA 10, BANDUNG 40132

2026

DAFTAR ISI

A. Deskripsi Persoalan.....	3
B. Penjelasan Implementasi.....	5
a. CNN.....	5
Konfigurasi dan Representasi Eksperimen.....	5
Loading Data dan Konstruksi Model Keras.....	6
Implementasi Scratch CNN.....	7
b. RNN.....	8
Konfigurasi, Data, dan Eksperimen RNN.....	8
Konstruksi Decoder Keras RNN.....	10
Implementasi Scratch RNN.....	10
Konversi Bobot dan Pipeline Scratch RNN.....	11
c. LSTM.....	12
Konfigurasi, Data, dan Eksperimen LSTM.....	12
Konstruksi Decoder Keras LSTM.....	13
Implementasi Scratch LSTM.....	14
Konversi Bobot dan Pipeline Scratch LSTM.....	15
C. Penjelasan Forward Propagation.....	17
a. CNN Forward Propagation.....	17
b. RNN Forward Propagation.....	18
c. LSTM Forward Propagation.....	20
D. Hasil Pengujian.....	22
a. CNN.....	22
Perbandingan Shared vs Non-Shared Parameter.....	22
Pengaruh jumlah layer konvolusi.....	24
Pengaruh banyak filter per layer konvolusi.....	24
Pengaruh ukuran filter per layer konvolusi.....	24
Pengaruh jenis pooling layer.....	24
Perbandingan Keras vs from scratch.....	24
b. Simple RNN dan LSTM untuk Image Captioning.....	24
Perbandingan jumlah layer: RNN.....	24
Perbandingan jumlah layer: LSTM.....	26

Perbandingan ukuran hidden state: RNN.....	27
Perbandingan ukuran hidden state: LSTM.....	28
Perbandingan RNN vs LSTM.....	30
Perbandingan Keras vs from scratch.....	31
Pengaruh panjang maksimum caption terhadap BLEU-4.....	32
E. Kesimpulan dan Saran.....	35
F. Pembagian Tugas.....	36
G. Referensi.....	37
H. Lampiran Form.....	38

A. Deskripsi Persoalan

Permasalahan pada tugas ini adalah mengimplementasikan dan menganalisis beberapa arsitektur neural network yang digunakan untuk pemrosesan data citra dan data sekuensial. Arsitektur yang dibahas meliputi Convolutional Neural Network (CNN), Simple Recurrent Neural Network (RNN), dan Long Short-Term Memory (LSTM). Setiap arsitektur memiliki mekanisme komputasi yang berbeda, sehingga tugas ini tidak hanya berfokus pada penggunaan library deep learning, tetapi juga pada pemahaman proses forward propagation dari masing-masing model.

Pada tugas ini, model dilatih menggunakan Keras untuk memperoleh bobot hasil pelatihan. Bobot tersebut kemudian digunakan kembali oleh implementasi from scratch yang dibuat menggunakan library dasar seperti NumPy. Implementasi from scratch bertujuan untuk memperlihatkan secara eksplisit bagaimana data diproses melalui setiap layer, mulai dari pembacaan input, operasi tensor atau matriks, penerapan fungsi aktivasi, hingga pembentukan output prediksi. Dengan demikian, implementasi ini dapat digunakan untuk membandingkan alur perhitungan manual dengan model referensi dari Keras.

Tugas ini terdiri dari dua konteks utama. Konteks pertama adalah klasifikasi gambar menggunakan CNN, yaitu model menerima sebuah gambar sebagai input dan memprediksi kelas dari gambar tersebut berdasarkan pola visual yang berhasil diekstraksi. Konteks kedua adalah image captioning menggunakan RNN dan LSTM, yaitu model menghasilkan caption atau deskripsi teks dari sebuah gambar dengan memanfaatkan feature vector dari CNN sebagai masukan ke decoder sekuensial. Kedua konteks ini menunjukkan penerapan neural network pada dua jenis data yang berbeda, yaitu data spasial berupa gambar dan data sekuensial berupa teks.

Selain implementasi model, tugas ini juga mencakup eksperimen terhadap beberapa variasi arsitektur dan hyperparameter. Eksperimen dilakukan untuk melihat bagaimana perubahan struktur model memengaruhi performa akhir. Pada bagian CNN, variasi dilakukan terhadap struktur layer

konvolusi, jumlah filter, ukuran kernel, dan jenis pooling. Pada bagian RNN/LSTM, variasi dilakukan terhadap struktur decoder untuk image captioning. Hasil eksperimen kemudian digunakan sebagai dasar analisis pada bagian hasil pengujian.

B. Penjelasan Implementasi

a. CNN

Konfigurasi dan Representasi Eksperimen

Implementasi CNN pada repository ini mencakup beberapa bagian utama, yaitu konfigurasi eksperimen, pembentukan kombinasi arsitektur, proses loading data, pembangunan model Keras, dan implementasi forward propagation secara scratch. Seluruh konfigurasi CNN didefinisikan dalam `src/cnn/config.py` menggunakan beberapa `dataclass`. Struktur ini digunakan agar parameter eksperimen dapat tersimpan rapi dan mudah diakses selama proses training maupun evaluasi.

Beberapa konfigurasi yang digunakan antara lain `CNNDataConfig`, `CNNTrainingConfig`, `CNNArchitectureDefaults`, `CNNExperimentGrid`, `CNNOutputConfig`, dan `CNNConfig`. `CNNDataConfig` menyimpan lokasi dataset dan properti citra, sedangkan `CNNTrainingConfig` menyimpan hyperparameter pelatihan seperti `batch_size`, `epochs`, `optimizer`, `learning_rate`, `loss`, dan `comparison_metric`. `CNNArchitectureDefaults` menyimpan parameter default arsitektur, `CNNExperimentGrid` menentukan ruang kombinasi eksperimen, `CNNOutputConfig` menyimpan lokasi output, dan `CNNConfig` menjadi objek konfigurasi utama untuk pipeline CNN.

Pada bagian konfigurasi, terdapat dua fungsi penting, yaitu `load_cnn_config()` dan `validate_cnn_config()`. Fungsi `load_cnn_config()` digunakan untuk membaca file JSON dan membentuk objek `CNNConfig`, sedangkan `validate_cnn_config()` digunakan untuk memeriksa validitas konfigurasi. Validasi ini mencakup format data, jumlah eksperimen, struktur folder dataset, dan kesesuaian parameter arsitektur. Dengan adanya validasi tersebut, kesalahan konfigurasi dapat diketahui lebih awal sebelum proses training dijalankan.

Setelah konfigurasi dibaca, kombinasi eksperimen CNN direpresentasikan melalui kelas `CNNExperiment` yang berada di `src/cnn/experiments.py`. Kelas ini menyimpan identitas satu kombinasi arsitektur, seperti `run_id`, `conv_layer_count`, `filters`, `kernel_sizes`, dan `pooling_type`. Method `to_dict()` digunakan untuk menyimpan metadata eksperimen ke dalam file JSON. Selain itu, fungsi `iter_cnn_experiments()` digunakan untuk membangkitkan seluruh kombinasi eksperimen berdasarkan grid yang telah ditentukan pada konfigurasi. Dengan cara ini, proses training dapat dilakukan untuk banyak variasi arsitektur tanpa harus menulis kombinasi secara manual satu per satu.

Loading Data dan Konstruksi Model Keras

Proses pelatihan CNN menggunakan kelas `CNNImageSequence` pada `src/cnn/training.py`. Kelas ini merupakan turunan dari `tf.keras.utils.Sequence`, sehingga dapat digunakan sebagai generator batch pada training Keras. Atribut yang disimpan di dalam kelas ini meliputi `records`, `config`, `batch_size`, `shuffle`, `rng`, dan `indexes`.

Beberapa method pentingnya adalah `__len__()` untuk menghitung jumlah batch, `__getitem__()` untuk mengambil satu batch gambar dan label, `on_epoch_end()` untuk mengacak urutan data setelah satu epoch selesai, serta properti `labels` untuk mengambil label ground truth dalam bentuk array NumPy. Kelas ini membantu proses pembacaan data agar lebih terstruktur, terutama ketika dataset tidak dimuat seluruhnya ke memori.

Arsitektur model CNN Keras dibangun di dalam `src/cnn/keras_models.py` melalui fungsi utama `build_cnn_model()`. Fungsi ini menerima objek `CNNConfig`, satu objek `CNNExperiment`, dan mode `parameter_sharing`. Secara umum, fungsi ini menyusun model dengan membuat input layer, menambahkan

convolution atau locally connected layer sesuai mode, menambahkan pooling layer, meratakan fitur menggunakan Flatten atau global pooling, menambahkan dense classifier, lalu menghasilkan output softmax untuk klasifikasi akhir.

Terdapat juga dua wrapper, yaitu `build_shared_cnn_model()` dan `build_nonshared_cnn_model()`. Keduanya memanggil fungsi inti yang sama, tetapi membedakan jenis layer konvolusinya. Pada mode shared digunakan `Conv2D`, sedangkan pada mode non-shared digunakan `LocallyConnected2D`.

Implementasi Scratch CNN

Selain implementasi menggunakan Keras, repository ini juga memiliki implementasi CNN scratch untuk menjalankan forward propagation secara manual. Bagian ini berada di dua file utama, yaitu `src/cnn/scratch_model.py` dan `src/cnn/scratch_layers.py`. Kelas utama pada level model adalah `ScratchCNNModel`, yang menyimpan atribut `layers` dan `name`. Method utamanya adalah `forward(x)`, yang menjalankan input secara berurutan melewati seluruh layer scratch. Struktur ini dibuat linear agar alur eksekusinya mirip dengan model Keras.

Forward pass pada layer scratch dipisahkan ke dalam beberapa kelas. `Conv2DLayer` digunakan untuk melakukan konvolusi 2D dengan shared weights, sedangkan `LocallyConnected2DLayer` digunakan untuk melakukan konvolusi tanpa weight sharing. `Pooling2DLayer` digunakan untuk menjalankan max pooling atau average pooling, `GlobalPooling2DLayer` digunakan untuk merangkum fitur spasial menjadi vektor global, `FlattenLayer` digunakan untuk mengubah tensor menjadi vektor datar, dan `DenseLayer` digunakan untuk menghitung transformasi linear serta aktivasi.

Setiap kelas hanya menangani satu operasi utama agar implementasi lebih modular dan mudah diperiksa. Selain itu, terdapat

fungsi bantu seperti `apply_activation()`, `_pad_input()`, `_patch_windows()`, dan `_output_size()` yang mendukung perhitungan numerik, terutama pada operasi konvolusi dan pooling.

Agar model scratch dapat menggunakan bobot hasil training Keras, digunakan fungsi `build_scratch_model_from_keras()` pada `src/cnn/scratch_model.py`. Fungsi ini membaca layer satu per satu dari model Keras, lalu mengubahnya menjadi objek layer scratch yang setara. Dengan pendekatan ini, model scratch tidak perlu dilatih dari awal. Model scratch cukup menggunakan bobot yang sudah diperoleh dari training Keras, lalu menjalankan forward pass manual untuk kebutuhan verifikasi dan perbandingan hasil.

b. RNN

Konfigurasi, Data, dan Eksperimen RNN

Pipeline RNN pada image captioning digunakan sebagai decoder sekuensial yang memproses representasi gambar dan urutan token caption. Konfigurasi captioning didefinisikan di `src/captioning/config.py` dengan struktur dataclass yang mencakup konfigurasi data, teks, encoder, training, eksperimen, dan output. Beberapa konfigurasi yang digunakan antara lain `CaptioningDataConfig`, `CaptioningTextConfig`, `CaptioningEncoderConfig`, `CaptioningTrainingConfig`, `CaptioningExperimentGrid`, `CaptioningOutputConfig`, dan `CaptioningConfig`.

`CaptioningDataConfig` menyimpan lokasi data gambar dan caption. `CaptioningTextConfig` menyimpan parameter preprocessing teks, seperti `start_token`, `end_token`, `pad_token`, `oov_token`, `min_word_frequency`, `max_caption_length`, dan `embedding_dim`. `CaptioningEncoderConfig` menyimpan konfigurasi encoder CNN, sedangkan `CaptioningTrainingConfig` menyimpan hyperparameter pelatihan. `CaptioningExperimentGrid` menyimpan kombinasi

eksperimen decoder, `CaptioningOutputConfig` menyimpan lokasi output, dan `CaptioningConfig` menjadi objek konfigurasi utama pipeline captioning.

Pada bagian konfigurasi, fungsi `load_captioning_config()` digunakan untuk membaca konfigurasi dan mengubah path relatif menjadi path absolut berdasarkan root repository. Setelah itu, `validate_captioning_config()` digunakan untuk mengecek apakah backbone encoder yang dipakai valid dan apakah seluruh file data yang dibutuhkan tersedia. Validasi ini diperlukan karena pipeline captioning bergantung pada data gambar, file caption, file split, dan konfigurasi encoder.

Data caption direpresentasikan di `src/captioning/data.py` melalui beberapa dataclass utama, yaitu `CaptionRecord`, `SplitCaptionDataset`, `CaptionDataset`, dan `Vocabulary`. `CaptionRecord` digunakan untuk menyimpan pasangan gambar dan satu caption mentah. `SplitCaptionDataset` menyimpan seluruh record dalam satu split beserta daftar `image_ids`, sedangkan `CaptionDataset` menggabungkan split train, validation, dan test ke dalam satu objek. Sementara itu, `Vocabulary` menyimpan peta token-ke-id dan id-ke-token, termasuk token khusus seperti `<pad>`, `<start>`, `<end>`, dan `<unk>`.

Beberapa fungsi penting pada bagian data adalah `load_caption_dataset()`, `load_caption_lookup()`, dan `build_split_dataset()`. Bagian ini menjadi fondasi untuk preprocessing dan evaluasi captioning karena seluruh caption ground truth dibaca dari struktur data tersebut.

Eksperimen RNN direpresentasikan oleh kelas `CaptionExperiment` dalam `src/captioning/training.py`. Kelas ini menyimpan beberapa atribut utama, yaitu `recurrent_type`, `layer_count`, dan `hidden_size`. Untuk eksperimen RNN, atribut `recurrent_type` berisi jenis decoder RNN. Selain itu, kelas ini juga

memiliki properti `run_id` yang secara otomatis membentuk nama eksperimen, misalnya `rnn_preinject_layers2_hidden128`. Penamaan ini digunakan secara konsisten untuk folder model dan file hasil evaluasi.

Konstruksi Decoder Keras RNN

Decoder RNN dibangun melalui fungsi `build_caption_decoder()` di `src/captioning/keras_models.py`. Fungsi ini menerima konfigurasi, jenis recurrent layer, jumlah layer, hidden size, dan ukuran vocabulary. Untuk model RNN, recurrent layer yang digunakan adalah `SimpleRNN`.

Arsitektur decoder mengikuti pola pre-inject. Fitur gambar dimasukkan sebagai `feature_inputs`, lalu diproyeksikan menggunakan `Dense` ke dimensi embedding. Hasil proyeksi tersebut diubah menjadi satu timestep dengan `Reshape`, kemudian digabungkan dengan input caption yang sudah melalui layer `Embedding`. Urutan gabungan tersebut diproses oleh stack `SimpleRNN`, lalu timestep fitur awal dibuang menggunakan `Lambda`. Hasil akhirnya dipetakan ke distribusi kata menggunakan `TimeDistributed(Dense(..., softmax))`.

Terdapat juga fungsi bantu `_apply_recurrent_stack()` untuk menambahkan beberapa layer RNN secara berulang, serta `_compile_model()` untuk memasang optimizer, loss, dan metrik. Dengan struktur ini, variasi jumlah layer dan hidden size pada decoder RNN dapat diuji menggunakan pola eksperimen yang sama.

Implementasi Scratch RNN

Implementasi scratch RNN dipisah ke dalam beberapa komponen dasar. `EmbeddingLayer` berada di `src/captioning/scratch/embedding.py` dan menyimpan atribut `weights` berupa matriks embedding. Method `forward(token_ids)` digunakan untuk mengambil vektor embedding berdasarkan token

tertentu. Operasi ini menjadi dasar bagi decoder karena setiap token harus diubah lebih dulu menjadi representasi vektor.

`DenseLayer` berada di `src/captioning/scratch/dense.py` dan memiliki atribut `kernel`, `bias`, serta `activation`. Method `forward(inputs)` menghitung transformasi linear $xW + b$, lalu menerapkan aktivasi jika diperlukan. Layer ini digunakan sebagai projection layer dari fitur gambar ke ruang embedding, sekaligus sebagai output layer untuk menghasilkan distribusi probabilitas vocabulary.

Komponen recurrent utama untuk model ini adalah `SimpleRNNCell` yang berada di `src/captioning/scratch/rnn.py`. Atribut yang disimpan adalah `kernel`, `recurrent_kernel`, dan `bias`. Method `step(x_t, h_t)` digunakan untuk melakukan satu langkah pembaruan hidden state dengan rumus RNN sederhana berbasis `tanh`. Karena RNN hanya menyimpan hidden state, prosesnya lebih sederhana dibandingkan LSTM.

Pada level arsitektur penuh, decoder scratch RNN dibungkus dalam kelas `ScratchRNNDecoder` di `src/captioning/scratch/model.py`. Kelas ini memiliki atribut `projection`, `embedding`, `recurrent`, `output`, `start_id`, `end_id`, dan `hidden_size`. Method yang tersedia adalah `generate(feature, max_length)` untuk menghasilkan caption dari satu feature vector dan `generate_batch(features, max_length)` untuk menghasilkan caption dari banyak gambar sekaligus.

Alur kerjanya dimulai dari feature gambar yang diproyeksikan ke ruang embedding, lalu digunakan sebagai timestep awal. Setelah itu, decoder memulai token dengan `<start>`, memprediksi token berikutnya menggunakan `argmax`, dan mengulangi proses sampai mencapai `<end>` atau panjang maksimum. Pada RNN, state yang dibawa dari satu timestep ke timestep berikutnya hanya berupa hidden state.

Konversi Bobot dan Pipeline Scratch RNN

Agar decoder scratch RNN dapat menggunakan bobot dari model Keras, digunakan fungsi `build_scratch_decoder_from_keras()` di `src/captioning/scratch/weights.py`. Fungsi ini membaca layer projection, embedding, recurrent pertama, dan output dari model Keras yang sudah dilatih. Bobot tersebut kemudian dikonversi menjadi `DenseLayer`, `EmbeddingLayer`, `SimpleRNNCell`, dan `ScratchRNNDecoder`.

Dengan pendekatan ini, implementasi scratch RNN dapat menjalankan inference menggunakan bobot hasil training yang sama. Artinya, perbandingan Keras dan scratch dilakukan pada bobot yang identik, bukan pada model yang berbeda.

Di atas decoder scratch, terdapat pipeline tambahan pada `src/captioning/scratch_pipeline.py`. Fungsi utama pada bagian ini adalah `load_scratch_decoder()`, `generate_scratch_caption()`, dan `generate_scratch_captions()`. `load_scratch_decoder()` digunakan untuk memuat model Keras dari disk, lalu membangunnya kembali menjadi decoder scratch. `generate_scratch_caption()` digunakan untuk menghasilkan caption dari satu gambar, sedangkan `generate_scratch_captions()` digunakan untuk menghasilkan caption dari satu batch gambar. Keduanya mengubah token id hasil prediksi menjadi kalimat caption akhir melalui fungsi `decode_token_ids()`.

c. LSTM

Konfigurasi, Data, dan Eksperimen LSTM

Pipeline LSTM pada image captioning memiliki fondasi konfigurasi dan data yang sama dengan pipeline RNN. Konfigurasi captioning tetap didefinisikan di `src/captioning/config.py` menggunakan beberapa dataclass, yaitu `CaptioningDataConfig`, `CaptioningTextConfig`, `CaptioningEncoderConfig`, `CaptioningTrainingConfig`, `CaptioningExperimentGrid`, `CaptioningOutputConfig`, dan

`CaptioningConfig`. Perbedaannya terletak pada jenis recurrent layer yang digunakan dalam eksperimen decoder.

Pada eksperimen LSTM, kelas `CaptionExperiment` tetap digunakan untuk merepresentasikan kombinasi model. Atribut yang disimpan adalah `recurrent_type`, `layer_count`, dan `hidden_size`. Untuk eksperimen LSTM, atribut `recurrent_type` berisi jenis decoder LSTM. Properti `run_id` secara otomatis membentuk nama eksperimen, misalnya `lstm_preinject_layers2_hidden128`. Penamaan ini digunakan untuk membedakan hasil eksperimen LSTM dari eksperimen RNN pada folder model dan file evaluasi.

Fungsi `iter_caption_experiments()` digunakan untuk menghasilkan seluruh kombinasi eksperimen berdasarkan grid konfigurasi. Dengan fungsi ini, variasi model LSTM dapat dibentuk dari kombinasi jumlah layer dan ukuran hidden state yang telah ditentukan sebelumnya. Karena data dan vocabulary yang digunakan sama dengan RNN, perbandingan antara RNN dan LSTM dapat dilakukan secara lebih adil.

Konstruksi Decoder Keras LSTM

Decoder LSTM juga dibangun melalui fungsi `build_caption_decoder()` di `src/captioning/keras_models.py`. Fungsi ini menerima konfigurasi, jenis recurrent layer, jumlah layer, hidden size, dan ukuran vocabulary. Untuk model LSTM, recurrent layer yang digunakan adalah LSTM.

Arsitektur decoder tetap mengikuti pola pre-inject. Fitur gambar dimasukkan sebagai `feature_inputs`, lalu diproyeksikan menggunakan `Dense` ke dimensi embedding. Hasil proyeksi tersebut diubah menjadi satu timestep dengan `Reshape`, kemudian digabungkan dengan input caption yang sudah melalui layer `Embedding`. Urutan gabungan tersebut diproses oleh stack LSTM, lalu timestep fitur awal dibuang menggunakan

Lambda. Hasil akhirnya dipetakan ke distribusi kata menggunakan `TimeDistributed(Dense(..., softmax))`.

Fungsi bantu `_apply_recurrent_stack()` digunakan untuk menambahkan beberapa layer LSTM secara berulang, sedangkan `_compile_model()` digunakan untuk memasang optimizer, loss, dan metrik. Dengan struktur ini, model LSTM dapat diuji dalam beberapa variasi arsitektur tanpa mengubah alur utama pipeline.

Implementasi Scratch LSTM

Komponen dasar pada scratch LSTM tetap menggunakan `EmbeddingLayer` dan `DenseLayer`. `EmbeddingLayer` digunakan untuk mengubah token id menjadi vektor embedding, sedangkan `DenseLayer` digunakan sebagai projection layer dan output layer. Projection layer memetakan fitur gambar ke ruang embedding, sementara output layer menghasilkan distribusi probabilitas vocabulary.

Komponen recurrent utama untuk model ini adalah `LSTMCell` yang berada di `src/captioning/scratch/lstm.py`. Atribut yang disimpan oleh kelas ini adalah `kernel`, `recurrent_kernel`, `bias`, dan `hidden_units`. Method `step(x_t, h_t, c_t)` menghitung empat gate LSTM, yaitu input gate, forget gate, candidate state, dan output gate. Setelah itu, kelas ini memperbarui `cell state` dan `hidden state`.

Berbeda dengan RNN, LSTM membawa dua state sekaligus pada setiap timestep. Hidden state digunakan sebagai representasi keluaran sementara, sedangkan cell state menyimpan informasi jangka panjang yang dapat dipertahankan atau dilupakan melalui mekanisme gate. Implementasi sigmoid pada bagian ini diberi clipping agar komputasi lebih stabil ketika menghadapi nilai besar.

Pada level arsitektur penuh, decoder scratch LSTM dibungkus dalam kelas `ScratchLSTMDecoder` di `src/captioning/scratch/model.py`. Kelas ini memiliki atribut `projection`, `embedding`, `recurrent`, `output`, `start_id`, `end_id`, dan

`hidden_units`. Method yang tersedia adalah `generate(feature, max_length)` untuk menghasilkan caption dari satu feature vector dan `generate_batch(features, max_length)` untuk menghasilkan caption dari banyak gambar sekaligus.

Alur kerja decoder LSTM dimulai dari feature gambar yang diproyeksikan ke ruang embedding, lalu digunakan sebagai timestep awal. Setelah itu, decoder memulai token dengan `<start>`, memprediksi token berikutnya menggunakan `argmax`, dan mengulangi proses sampai mencapai `<end>` atau panjang maksimum. Selama proses tersebut, LSTM memperbarui hidden state dan cell state pada setiap timestep.

Konversi Bobot dan Pipeline Scratch LSTM

Agar decoder scratch LSTM dapat menggunakan bobot dari model Keras, digunakan fungsi `build_scratch_decoder_from_keras()` di `src/captioning/scratch/weights.py`. Fungsi ini membaca layer projection, embedding, recurrent pertama, dan output dari model Keras yang sudah dilatih. Bobot tersebut kemudian dikonversi menjadi `DenseLayer`, `EmbeddingLayer`, `LSTMCell`, dan `ScratchLSTMDecoder`.

Dengan pendekatan ini, implementasi scratch LSTM dapat menjalankan inference menggunakan bobot hasil training yang sama. Perbandingan antara model Keras dan scratch dilakukan pada bobot yang identik, sehingga hasil yang diperoleh dapat digunakan untuk memverifikasi kesesuaian implementasi manual.

Pipeline tambahan untuk menjalankan scratch LSTM berada di `src/captioning/scratch_pipeline.py`. Fungsi utama pada bagian ini adalah `load_scratch_decoder()`, `generate_scratch_caption()`, dan `generate_scratch_captions()`. `load_scratch_decoder()` digunakan untuk memuat model Keras dari disk, lalu membangunnya kembali menjadi decoder scratch. `generate_scratch_caption()` digunakan untuk menghasilkan caption dari satu gambar, sedangkan `generate_scratch_captions()` digunakan untuk menghasilkan

caption dari satu batch gambar. Hasil prediksi berupa token id kemudian diubah menjadi kalimat caption akhir melalui fungsi `decode_token_ids()`.

C. Penjelasan Forward Propagation

a. CNN Forward Propagation

Proses forward propagation pada CNN dimulai ketika input gambar dimasukkan ke dalam model. Input gambar direpresentasikan sebagai tensor dengan bentuk (N, H, W, C) , dengan N sebagai jumlah data dalam batch, H dan W sebagai tinggi dan lebar gambar, serta C sebagai jumlah channel warna. Pada implementasi ini, format data yang digunakan adalah channels last, sehingga channel warna RGB berada pada dimensi terakhir.

Tahap pertama dalam forward propagation adalah pemrosesan melalui layer konvolusi. Pada layer Conv2D, kernel berukuran tertentu digeser melintasi dimensi spasial gambar. Pada setiap posisi, model mengambil patch kecil dari input, kemudian menghitung dot product antara patch tersebut dan bobot kernel. Hasil dot product ditambahkan dengan bias, lalu dilewatkan ke fungsi aktivasi. Karena Conv2D menggunakan shared parameter, kernel yang sama digunakan pada seluruh posisi spasial input. Mekanisme ini memungkinkan model mendeteksi pola visual yang sama di berbagai posisi gambar.

Untuk variasi non-shared parameter, operasi dilakukan menggunakan layer LocallyConnected2D. Secara umum, layer ini juga mengambil patch dari input dan menghitung hasil perkalian antara patch dan bobot. Perbedaannya, bobot yang digunakan pada setiap posisi output tidak dibagikan ke posisi lain. Dengan kata lain, setiap lokasi spasial memiliki parameter sendiri. Hal ini membuat operasi forward propagation pada LocallyConnected2D mirip dengan konvolusi, tetapi tanpa konsep parameter sharing.

Setelah melewati layer konvolusi atau locally connected, output diproses oleh fungsi aktivasi ReLU. Fungsi ReLU mengubah nilai negatif menjadi nol dan mempertahankan nilai positif. Aktivasi ini digunakan untuk memberikan non-linearitas pada model, sehingga model tidak hanya merepresentasikan transformasi linear dari input.

Tahap berikutnya adalah pooling. Pooling digunakan untuk mereduksi ukuran spasial feature map sambil mempertahankan informasi penting. Pada max pooling, nilai terbesar pada setiap window dipilih sebagai output. Pada average pooling, output dihitung sebagai rata-rata nilai pada window tersebut. Operasi pooling dilakukan secara terpisah pada setiap channel feature map.

Setelah feature map melewati seluruh blok konvolusi dan pooling, output kemudian diubah menjadi vektor satu dimensi menggunakan layer Flatten. Proses flatten dilakukan dengan menyusun seluruh nilai pada feature map ke dalam satu vektor untuk setiap data dalam batch. Vektor ini kemudian menjadi input bagi dense layer.

Dense layer melakukan operasi perkalian matriks antara input dan bobot, kemudian menambahkan bias. Hasil dari dense layer tersembunyi dilewatkan ke fungsi aktivasi ReLU. Pada layer output, hasil linear dari dense layer terakhir dilewatkan ke fungsi softmax. Fungsi softmax mengubah nilai output menjadi distribusi probabilitas untuk seluruh kelas. Setiap elemen output menunjukkan probabilitas model terhadap salah satu kelas yang mungkin.

Pada implementasi from scratch, setiap layer memiliki method `forward(...)` yang menerima input NumPy array dan menghasilkan output NumPy array. Model `ScratchCNNModel` menjalankan proses forward propagation dengan meneruskan output dari satu layer sebagai input ke layer berikutnya secara berurutan. Implementasi ini juga mendukung batch inference, karena setiap layer menerima input batch dan mempertahankan dimensi batch hingga output akhir.

b. RNN Forward Propagation

Pada decoder RNN, informasi yang dibawa dari satu timestep ke timestep berikutnya hanya berupa **hidden state**. Hidden state ini berfungsi sebagai ringkasan konteks urutan token yang telah diproses sejauh itu.

Misalkan:

- x_t adalah input pada timestep ke- t
- h_t adalah hidden state pada timestep ke- t
- W_x adalah bobot input
- W_h adalah bobot rekuren
- b adalah bias

Maka satu langkah forward pada RNN dituliskan sebagai:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

Pada implementasi tugas ini, langkah pertama tidak langsung dimulai dari token kata, tetapi dari **feature gambar**. Feature vector hasil encoder CNN lebih dulu dilewatkan ke projection layer agar dimensinya sesuai dengan embedding token. Hasil proyeksi ini kemudian diperlakukan sebagai input awal untuk membentuk hidden state pertama.

Setelah hidden state awal terbentuk, model mulai masuk ke proses decoding kata. Token `<start>` diubah dulu menjadi embedding vector melalui embedding layer. Embedding ini menjadi input `xtx_txt` pada timestep berikutnya. Hidden state yang sudah ada lalu digabung dengan input baru melalui persamaan RNN di atas untuk menghasilkan hidden state baru. Dari hidden state tersebut, output layer menghitung distribusi probabilitas terhadap seluruh vocabulary. Token dengan probabilitas tertinggi dipilih sebagai prediksi berikutnya.

Proses ini diulang terus. Token hasil prediksi pada satu langkah akan menjadi input untuk langkah setelahnya. Dengan kata lain, model berjalan secara autoregresif. Caption dibentuk sedikit demi sedikit sampai token `<end>` muncul atau panjang caption mencapai batas maksimum.

Dari sisi implementasi scratch, alur ini dijalankan di dalam `SimpleRNNCell.step()` dan `ScratchRNNDecoder.generate_batch()`. `SimpleRNNCell.step()` menangani pembaruan hidden state per timestep, sedangkan `ScratchRNNDecoder` mengatur keseluruhan proses decoding, mulai dari inisialisasi state, prediksi token, sampai penghentian caption.

Kelebihan mekanisme ini adalah bentuknya sederhana dan mudah diimplementasikan. Namun ada kelemahan yang cukup mendasar. Karena seluruh konteks urutan diringkas hanya dalam satu hidden state, informasi dari timestep awal bisa semakin pudar saat urutan menjadi lebih panjang. Pada tugas captioning, kondisi ini dapat membuat model kesulitan menjaga hubungan antar kata saat caption mulai kompleks.

c. LSTM Forward Propagation

Berbeda dari RNN biasa, LSTM menyimpan dua komponen state sekaligus, yaitu **hidden state** dan **cell state**. Hidden state dipakai untuk menghasilkan output saat ini, sedangkan cell state berfungsi sebagai jalur memori yang lebih stabil untuk membawa informasi lintas timestep.

Pada setiap langkah, LSTM menghitung empat komponen utama:

- input gate
- forget gate
- candidate state
- output gate

Jika x_t menyatakan input saat ini, h_{t-1} hidden state sebelumnya, dan c_{t-1} cell state sebelumnya, maka perhitungannya dapat diringkas sebagai:

$$i_t = \sigma(x_t W_i + h_{t-1} U_i + b_i)$$

$$f_t = \sigma(x_t W_f + h_{t-1} U_f + b_f)$$

$$\tilde{c}_t = \tanh(x_t W_c + h_{t-1} U_c + b_c)$$

$$o_t = \sigma(x_t W_o + h_{t-1} U_o + b_o)$$

Setelah itu, cell state dan hidden state diperbarui dengan:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

Inti dari mekanisme ini ada pada gate. **Forget gate** menentukan bagian memori lama yang tetap dipertahankan, **input gate** mengatur informasi baru yang akan ditambahkan, dan **output gate** menentukan bagian mana dari cell state yang akan dikeluarkan sebagai hidden state.

Karena ada mekanisme pengendali seperti ini, LSTM lebih mampu menjaga informasi penting dalam urutan yang lebih panjang.

Dalam pipeline captioning pada tugas ini, langkah awalnya tetap sama seperti RNN. Feature gambar dari encoder CNN diproyeksikan lebih dulu ke dimensi embedding, lalu dipakai sebagai input awal untuk memperbarui state. Bedanya, saat langkah pertama ini dilakukan, LSTM tidak hanya menghasilkan hidden state awal, tetapi juga cell state awal. Kedua state inilah yang kemudian dibawa ke langkah-langkah decoding berikutnya.

Setelah itu token `<start>` diubah ke embedding vector, dimasukkan ke sel LSTM, lalu model menghitung distribusi probabilitas kata berikutnya. Token dengan probabilitas tertinggi dipilih dan dipakai lagi sebagai input pada langkah berikutnya. Prosesnya tetap autoregresif, tetapi sekarang setiap langkah mempertahankan dua jenis state sekaligus.

Pada implementasi scratch, mekanisme ini dikerjakan oleh `LSTMCell.step()` dan `ScratchLSTMDecoder.generate_batch()`. `LSTMCell.step()` menghitung seluruh gate serta pembaruan hidden state dan cell state, sedangkan `ScratchLSTMDecoder` mengatur alur caption generation secara penuh.

D. Hasil Pengujian

a. CNN

Perbandingan Shared vs Non-Shared Parameter

Pengujian shared dan non-shared parameter dilakukan menggunakan arsitektur terbaik dari eksperimen CNN shared parameter. Arsitektur tersebut terdiri dari 2 layer konvolusi dengan jumlah filter 32 dan 64, ukuran kernel 3×3 dan 3×3, serta max pooling. Pada model shared parameter, layer yang digunakan adalah Conv2D, sedangkan pada model non-shared parameter seluruh Conv2D diganti dengan LocallyConnected2D.

Model	Jumlah Parameter	Akurasi Validasi	Macro F1 Validasi
Shared parameter	7.393.094	0.7855	0.7862
Non-shared parameter	90.422.214	0.6750	0.6745

Hasil pengujian menunjukkan bahwa model shared parameter memperoleh performa yang lebih baik dibandingkan model non-shared parameter. Model shared menghasilkan macro F1 validasi sebesar 0,7862, sedangkan model non-shared hanya memperoleh 0,6745. Perbedaan ini cukup besar meskipun model non-shared memiliki jumlah parameter jauh lebih banyak, yaitu sekitar 90 juta parameter dibandingkan 7,3 juta parameter pada model shared.

Hal ini menunjukkan bahwa parameter sharing pada CNN berperan penting dalam meningkatkan efisiensi dan kemampuan generalisasi model. Dengan menggunakan kernel yang sama pada berbagai posisi spasial, model shared dapat mengenali pola visual yang sama meskipun muncul di lokasi berbeda pada gambar. Sebaliknya,

model non-shared memiliki parameter berbeda untuk setiap posisi, sehingga kapasitas model menjadi jauh lebih besar dan lebih rentan mempelajari pola yang terlalu spesifik terhadap data latih.

Pengaruh Banyak Filter per Layer Konvolusi

Dua variasi jumlah layer konvolusi diuji, yaitu model dengan 1 layer konvolusi dan model dengan 2 layer konvolusi. Perbandingan dilakukan berdasarkan rata-rata macro F1 validasi dari seluruh konfigurasi yang memiliki jumlah layer yang sama.

Jumlah Layer Konvolusi	Rata-rata Macro F1	Macro F1
1 Layer	0.6397	0.6809
2 Layer	0.7463	0.7862

Hasil pengujian menunjukkan bahwa penggunaan 2 layer konvolusi memberikan peningkatan performa yang cukup signifikan dibandingkan 1 layer konvolusi. Rata-rata macro F1 meningkat dari 0,6397 menjadi 0,7463. Selain itu, konfigurasi terbaik secara keseluruhan juga berasal dari model dengan 2 layer konvolusi.

Peningkatan ini menunjukkan bahwa layer konvolusi tambahan membantu model membangun representasi fitur yang lebih kompleks. Layer awal dapat menangkap pola sederhana seperti tepi, warna, dan tekstur, sedangkan layer berikutnya dapat menggabungkan pola tersebut menjadi fitur yang lebih bermakna untuk klasifikasi gambar.

Pengaruh Banyak Filter per Layer Konvolusi

Variasi jumlah filter diuji menggunakan beberapa profil filter, yaitu 16, 32, 16-32, dan 32-64. Profil dengan satu angka digunakan pada arsitektur 1 layer konvolusi, sedangkan profil dengan dua angka digunakan pada arsitektur 2 layer konvolusi.

Profil Filter	Rata-rata Macro F1	Macro F1 Terbaik
16	0.6437	0.6636
32	0.6357	0.6809
16-32	0.7448	0.7663
32-64	0.7479	0.7862

Berdasarkan hasil pengujian, konfigurasi dengan profil filter 32-64 memperoleh performa terbaik dengan macro F1 maksimum sebesar 0,7862. Profil 16-32 juga menghasilkan performa yang cukup tinggi, yaitu rata-rata macro F1 sebesar 0,7448. Kedua profil ini lebih baik dibandingkan konfigurasi satu layer dengan 16 atau 32 filter.

Hal ini menunjukkan bahwa peningkatan jumlah filter dapat membantu model menangkap variasi fitur visual yang lebih beragam. Namun, peningkatan filter terlihat paling efektif ketika dikombinasikan dengan jumlah layer konvolusi yang lebih banyak. Dengan dua layer konvolusi, model memiliki ruang representasi yang lebih besar untuk mempelajari fitur bertingkat dari gambar.

Pengaruh Ukuran Filter per Layer Konvolusi

Ukuran kernel yang diuji adalah 3×3 , 5×5 , $3 \times 3 - 3 \times 3$, dan $5 \times 5 - 3 \times 3$. Perbandingan dilakukan untuk melihat pengaruh ukuran receptive field terhadap performa klasifikasi.

Profil Filter	Rata-rata Macro F1	Macro F1 Terbaik
3×3	0.6421	0.6809
5×5	0.6372	0.6538
$3 \times 3 - 3 \times 3$	0.7651	0.7862
$5 \times 5 - 3 \times 3$	0.7276	0.7468

Hasil pengujian menunjukkan bahwa konfigurasi kernel $3 \times 3 - 3 \times 3$ menghasilkan performa terbaik, dengan rata-rata macro F1 sebesar 0,7651 dan macro F1 terbaik sebesar 0,7862. Konfigurasi yang menggunakan kernel 5×5 cenderung menghasilkan performa lebih rendah dibandingkan konfigurasi 3×3 .

Kernel 3×3 lebih efektif pada eksperimen ini karena mampu menangkap pola lokal dengan jumlah parameter yang lebih efisien. Selain itu, penggunaan dua layer 3×3 secara berturut-turut memungkinkan model membangun representasi fitur yang lebih kompleks secara bertahap, tanpa langsung menggunakan kernel besar yang dapat meningkatkan jumlah parameter dan risiko overfitting.

Pengaruh Jenis Pooling Layer

Dua jenis pooling diuji pada eksperimen ini, yaitu max pooling dan average pooling. Perbandingan dilakukan berdasarkan rata-rata macro F1 dari seluruh konfigurasi yang menggunakan masing-masing jenis pooling.

Jenis Pooling	Rata-rata Macro F1	Macro F1 Terbaik
Average Pooling	0.6803	0.7541
Max Pooling	0.7057	0.7862

Hasil pengujian menunjukkan bahwa max pooling memberikan performa lebih baik dibandingkan average pooling. Rata-rata macro F1 pada max pooling adalah 0,7057, sedangkan average pooling memperoleh 0,6803. Konfigurasi terbaik secara keseluruhan juga menggunakan max pooling.

Max pooling cenderung lebih sesuai untuk task klasifikasi gambar karena mempertahankan aktivasi paling kuat pada setiap area feature map. Aktivasi yang kuat biasanya merepresentasikan fitur visual penting seperti tepi, tekstur, atau pola objek tertentu. Sebaliknya, average pooling menghitung rata-rata nilai dalam window, sehingga fitur dominan dapat menjadi kurang menonjol.

Perbandingan Keras vs From Scratch

Perbandingan Keras dan from scratch dilakukan menggunakan arsitektur shared parameter terbaik. Bobot hasil pelatihan Keras dimuat ke dalam implementasi forward propagation from scratch berbasis NumPy. Pengujian dilakukan pada data test untuk memastikan bahwa implementasi from scratch menghasilkan output yang konsisten dengan Keras.

Jenis Pooling	Rata-rata Macro F1	Macro F1 Terbaik
Keras	0.7845	0.7833
From Scratch	0.7845	0.7833

Hasil pengujian menunjukkan bahwa implementasi Keras dan from scratch menghasilkan macro F1 dan akurasi test yang sama. Selain itu, prediction agreement antara kedua implementasi mencapai 1,0, yang berarti seluruh prediksi kelas yang dihasilkan oleh model from scratch sama dengan prediksi dari model Keras.

Selisih maksimum probabilitas output antara Keras dan from scratch adalah $1,97e-06$. Nilai ini sangat kecil dan masih wajar karena perbedaan detail numerik pada operasi floating point. Dengan demikian, implementasi forward propagation CNN from scratch dapat dianggap sudah sesuai dengan model referensi Keras.

b. Simple RNN dan LSTM untuk Image Captioning

Perbandingan Jumlah Layer: RNN

Jumlah layer	Hidden state	BLEU-4	METEOR	Waktu eksekusi (detik)
1	128	0.0573	0.3479	467.81
1	512	0.0473	0.3387	494.46
2	128	0.0605	0.3592	515.30
2	512	0.0379	0.3334	629.99
3	128	0.0454	0.3634	743.77
3	512	0.0473	0.3289	602.28

Pada model RNN, penambahan jumlah *layer* memengaruhi kemampuan model dalam membentuk representasi sekuensial. Secara umum, penambahan *layer* dapat meningkatkan kapasitas model karena setiap *layer* memiliki peran dalam memproses informasi dari tingkat representasi yang berbeda. *Layer* awal cenderung menangkap pola urutan sederhana, sedangkan *layer* berikutnya dapat mengolah pola

tersebut menjadi representasi yang lebih abstrak. Hal ini terlihat pada konfigurasi **2 layer dengan 128 hidden state**, yang menghasilkan BLEU-4 tertinggi sebesar **0.0605** dan METEOR sebesar **0.3592**.

Namun, peningkatan jumlah *layer* tidak selalu menghasilkan performa yang lebih baik. Ketika jumlah *layer* dinaikkan menjadi 3, nilai BLEU-4 justru menurun pada kedua variasi ukuran *hidden state*. Penurunan ini dapat dijelaskan melalui karakteristik dasar RNN yang memiliki keterbatasan dalam mempertahankan informasi pada urutan yang panjang. Pada RNN, informasi dari *timestep* sebelumnya hanya dibawa melalui *hidden state*, sehingga semakin panjang dan kompleks jalur komputasi yang dilalui, semakin besar kemungkinan informasi penting mengalami pelemahan.

Selain itu, penambahan *layer* juga meningkatkan kompleksitas model dan jumlah operasi yang harus dilakukan. Model dengan 3 *layer* harus mempelajari hubungan antar *token* secara temporal sekaligus memproses transformasi antar *layer* yang lebih banyak. Kondisi ini dapat membuat proses optimasi menjadi lebih sulit, terutama apabila jumlah data dan konfigurasi pelatihan belum cukup untuk mendukung model yang lebih kompleks. Oleh karena itu, konfigurasi dengan jumlah *layer* yang lebih besar tidak selalu memberikan peningkatan kualitas *caption*.

Nilai METEOR pada model **3 layer dengan 128 hidden state** memang menjadi yang tertinggi pada kelompok RNN, yaitu **0.3634**. Akan tetapi, nilai BLEU-4 pada konfigurasi tersebut lebih rendah dibandingkan model **2 layer dengan 128 hidden state**. Hal ini menunjukkan bahwa model 3 *layer* masih mampu menghasilkan kata-kata yang cukup relevan terhadap referensi, tetapi belum mampu mempertahankan struktur frasa yang sebaik model 2 *layer*. BLEU-4 lebih sensitif terhadap kesesuaian urutan *n-gram*, sedangkan METEOR lebih toleran terhadap variasi kecocokan kata. Dengan demikian, konfigurasi **2 layer dengan 128 hidden state** dapat dianggap sebagai konfigurasi yang paling seimbang untuk model RNN.

Perbandingan Jumlah Layer: LSTM

Jumlah layer	Hidden state	BLEU-4	METEOR	Waktu eksekusi (detik)
1	128	0.0557	0.3675	595.05
1	512	0.0545	0.3772	708.13
2	128	0.0679	0.3650	470.63
2	512	0.0653	0.3686	640.21
3	128	0.0629	0.3692	528.81
3	512	0.0635	0.3690	598.88

Pada model LSTM, pengaruh jumlah *layer* menunjukkan pola yang lebih stabil dibandingkan RNN. Hal ini berkaitan dengan karakteristik LSTM yang memiliki *cell state* dan mekanisme *gate* untuk mengatur aliran informasi pada setiap *timestep*. Mekanisme tersebut membantu model mempertahankan informasi yang relevan dan mengurangi risiko hilangnya informasi penting pada urutan yang lebih panjang. Oleh karena itu, LSTM cenderung lebih mampu menangani struktur sekuensial dibandingkan RNN sederhana.

Konfigurasi terbaik pada kelompok LSTM diperoleh oleh model **2 layer dengan 128 hidden state**, dengan BLEU-4 sebesar **0.0679** dan METEOR sebesar **0.3650**. Hasil ini menunjukkan bahwa penambahan *layer* dari 1 menjadi 2 memberikan peningkatan kapasitas yang bermanfaat. Dengan dua *layer*, model memiliki kemampuan yang lebih baik dalam membentuk representasi sekuensial yang bertingkat, sehingga hubungan antara fitur gambar dan urutan kata dapat dipelajari dengan lebih efektif.

Namun, ketika jumlah *layer* ditingkatkan menjadi 3, performa model tidak lagi mengalami peningkatan yang berarti. Hal ini menunjukkan bahwa tambahan kedalaman model tidak selalu memberikan kontribusi positif terhadap kualitas *caption*. Pada kondisi tertentu, model yang terlalu kompleks dapat menjadi kurang efisien karena jumlah parameter dan operasi meningkat. Jika kompleksitas tambahan tersebut tidak didukung oleh kebutuhan pola data yang sepadan, peningkatan arsitektur justru tidak menghasilkan perbaikan performa yang signifikan.

Selain menghasilkan nilai BLEU-4 terbaik, konfigurasi **2 layer dengan 128 hidden state** juga memiliki waktu eksekusi terendah pada kelompok LSTM, yaitu **470.63 detik**. Hal ini menunjukkan bahwa konfigurasi tersebut tidak hanya unggul dari sisi kualitas, tetapi juga lebih efisien secara komputasi. Dengan demikian, pada eksperimen ini, konfigurasi **2 layer dengan 128 hidden state** merupakan pilihan paling efektif untuk decoder LSTM.

Perbandingan Ukuran Hidden State: RNN

Jumlah <i>layer</i>	<i>Hidden</i> 128 BLEU-4	<i>Hidden</i> 512 BLEU-4	<i>Hidden</i> 128 METEOR	<i>Hidden</i> 512 METEOR
1	0.0573	0.0473	0.3479	0.3387
2	0.0605	0.0379	0.3592	0.3334
3	0.0454	0.0473	0.3634	0.3289

Secara teoritis, ukuran *hidden state* yang lebih besar memberikan ruang representasi yang lebih luas bagi model. Dengan jumlah unit yang lebih besar, model memiliki kapasitas lebih tinggi untuk menyimpan informasi pada setiap *timestep*. Namun, peningkatan kapasitas tersebut tidak selalu berdampak positif terhadap performa, terutama pada model

RNN yang memiliki keterbatasan dalam mengatur informasi jangka panjang.

Pada hasil eksperimen, sebagian besar konfigurasi RNN menunjukkan performa yang lebih baik ketika menggunakan *hidden state* berukuran **128** dibandingkan **512**. Hal ini terlihat pada model 1 *layer* dan 2 *layer*, di mana *hidden state* 128 menghasilkan nilai BLEU-4 dan METEOR yang lebih tinggi. Penurunan paling besar terjadi pada model 2 *layer*, yaitu dari BLEU-4 **0.0605** pada *hidden state* 128 menjadi **0.0379** pada *hidden state* 512.

Penurunan performa tersebut dapat disebabkan oleh meningkatnya jumlah parameter rekuren ketika ukuran *hidden state* diperbesar. RNN harus mempelajari hubungan temporal antar *token* sekaligus memperbarui *hidden state* pada setiap langkah. Ketika dimensi *hidden state* menjadi terlalu besar, proses optimasi dapat menjadi lebih sulit. Kapasitas model yang lebih besar juga dapat menyebabkan model kurang mampu melakukan generalisasi apabila data yang tersedia tidak cukup untuk mendukung kompleksitas tersebut.

Selain itu, RNN tidak memiliki mekanisme pengendali informasi seperti *gate* pada LSTM. Akibatnya, peningkatan ukuran *hidden state* tidak secara otomatis meningkatkan kemampuan model dalam menyimpan konteks. Ruang representasi yang lebih besar justru dapat membuat pembelajaran menjadi lebih tidak stabil apabila tidak diimbangi dengan mekanisme memori yang memadai. Berdasarkan hasil ini, *hidden state* **128** lebih sesuai untuk model RNN karena memberikan performa yang lebih konsisten dan efisien.

Perbandingan Ukuran Hidden State: LSTM

Jumlah <i>layer</i>	<i>Hidden</i> 128 BLEU-4	<i>Hidden</i> 512 BLEU-4	<i>Hidden</i> 128 METEOR	<i>Hidden</i> 512 METEOR
------------------------	--------------------------------	-----------------------------	--------------------------------	--------------------------

1	0.0557	0.0545	0.3675	0.3772
2	0.0679	0.0653	0.3650	0.3686
3	0.0629	0.0635	0.3692	0.3690

Pada model LSTM, pengaruh ukuran *hidden state* terlihat lebih kecil dibandingkan pada RNN. Hal ini menunjukkan bahwa LSTM lebih stabil ketika kapasitas model ditingkatkan. Stabilitas tersebut berasal dari mekanisme *cell state* dan *gate*, yang memungkinkan model mengatur informasi mana yang perlu dipertahankan, diperbarui, atau dikeluarkan sebagai representasi pada setiap *timestep*.

Hasil eksperimen menunjukkan bahwa *hidden state* 512 kadang menghasilkan nilai METEOR yang sedikit lebih tinggi, terutama pada konfigurasi 1 *layer* dan 2 *layer*. Hal ini mengindikasikan bahwa peningkatan kapasitas dapat membantu model memilih kata yang secara makna lebih dekat dengan referensi. Namun, peningkatan tersebut tidak selalu diikuti oleh perbaikan BLEU-4. Pada model 1 *layer* dan 2 *layer*, BLEU-4 tetap lebih tinggi ketika menggunakan *hidden state* 128.

Perbedaan ini dapat dijelaskan melalui karakteristik metrik evaluasi. BLEU-4 lebih menekankan kesesuaian urutan frasa atau *n-gram*, sedangkan METEOR lebih mempertimbangkan kecocokan kata secara lebih fleksibel. Oleh karena itu, model dengan *hidden state* 512 mungkin mampu menghasilkan pilihan kata yang relevan, tetapi belum tentu menghasilkan susunan frasa yang lebih dekat dengan *caption* referensi.

Dari sisi efisiensi, *hidden state* 128 juga lebih menguntungkan. Pada LSTM, setiap *timestep* melibatkan beberapa operasi *gate*, sehingga peningkatan ukuran *hidden state* akan meningkatkan biaya komputasi secara cukup besar. Karena peningkatan performa dari *hidden state* 512 tidak konsisten dan relatif kecil, konfigurasi *hidden state* **128** dapat

dianggap lebih efisien. Dengan demikian, pada eksperimen ini, *hidden state* **128** menjadi pilihan yang lebih tepat untuk model LSTM.

Perbandingan RNN vs LSTM

Decoder	Best Run	Jumlah Layer	Hidden State	BLEU-4	METEOR	Time (detik)
RNN	<i>rnn_preinject_layers2_hidden128</i>	2	128	0.0605	0.3592	515.30
LSTM	<i>lstm_preinject_layers2_hidden128</i>	2	128	0.0679	0.3650	470.63

Perbandingan antara model terbaik RNN dan LSTM menunjukkan bahwa LSTM memiliki performa yang lebih baik pada kedua metrik utama. Model RNN terbaik, yaitu *rnn_preinject_layers2_hidden128*, memperoleh BLEU-4 sebesar **0.0605** dan METEOR sebesar **0.3592**. Sementara itu, model LSTM terbaik, yaitu *lstm_preinject_layers2_hidden128*, memperoleh BLEU-4 sebesar **0.0679** dan METEOR sebesar **0.3650**. Perbedaan ini menunjukkan bahwa LSTM lebih mampu menghasilkan *caption* yang mendekati referensi, baik dari sisi struktur frasa maupun relevansi kata.

Keunggulan LSTM dapat dijelaskan melalui mekanisme internalnya. Pada RNN, informasi dari *timestep* sebelumnya hanya disimpan dalam *hidden state*. Mekanisme ini membuat RNN lebih rentan mengalami pelemahan informasi, terutama ketika urutan yang diproses semakin panjang. Dalam tugas *image captioning*, kondisi tersebut menjadi penting karena model harus mempertahankan hubungan antara

fitur gambar awal dan kata-kata yang dihasilkan sepanjang proses *decoding*.

Sebaliknya, LSTM memiliki *cell state* sebagai jalur memori tambahan. *Cell state* memungkinkan informasi penting dipertahankan lebih lama, sedangkan mekanisme *gate* mengatur informasi mana yang perlu disimpan, dilupakan, atau digunakan sebagai keluaran. Dengan struktur tersebut, LSTM lebih sesuai untuk tugas yang membutuhkan pemahaman urutan dan pemeliharaan konteks, termasuk *image captioning*.

Model LSTM terbaik juga memiliki waktu eksekusi yang lebih rendah dibandingkan model RNN terbaik. RNN terbaik membutuhkan waktu **515.30 detik**, sedangkan LSTM terbaik membutuhkan **470.63 detik**. Meskipun secara arsitektur LSTM umumnya lebih kompleks, waktu eksekusi dalam eksperimen ini juga dipengaruhi oleh faktor lain, seperti panjang *caption* yang dihasilkan, stabilitas proses *decoding*, serta kondisi evaluasi. Dengan mempertimbangkan kualitas dan waktu eksekusi, LSTM menjadi model yang lebih unggul pada eksperimen ini.

Perbandingan Keras vs from scratch

<i>Decoder</i>	Implementasi	BLEU-4	METEOR	<i>Time (detik)</i>
RNN	Keras	0.0573	0.3479	493.60
RNN	<i>Scratch</i>	0.0223	0.2966	1.90
LSTM	Keras	0.0557	0.3675	617.08
LSTM	<i>Scratch</i>	0.0214	0.3164	3.66

Perbandingan antara implementasi Keras dan *from scratch* menunjukkan adanya perbedaan performa yang cukup jelas. Pada RNN, implementasi Keras memperoleh BLEU-4 sebesar **0.0573** dan METEOR sebesar **0.3479**, sedangkan implementasi *scratch* memperoleh BLEU-4

sebesar **0.0223** dan METEOR sebesar **0.2966**. Pada LSTM, pola yang sama juga terlihat, yaitu implementasi Keras memperoleh BLEU-4 sebesar **0.0557** dan METEOR sebesar **0.3675**, sedangkan implementasi *scratch* memperoleh BLEU-4 sebesar **0.0214** dan METEOR sebesar **0.3164**.

Perbedaan tersebut dapat terjadi karena implementasi Keras menggunakan operasi yang telah dioptimalkan dan tervalidasi dengan baik. Keras/TensorFlow menangani berbagai aspek teknis, seperti operasi *tensor*, presisi numerik, bentuk data, serta integrasi antar *layer*. Sebaliknya, implementasi *from scratch* pada tugas ini merupakan rekonstruksi manual dari proses *forward propagation*. Walaupun struktur model dibuat setara, terdapat kemungkinan perbedaan kecil dalam urutan operasi, stabilitas numerik, atau cara pembacaan bobot.

Dalam tugas *captioning*, perbedaan kecil pada satu langkah prediksi dapat berdampak besar terhadap keluaran akhir. Hal ini disebabkan oleh sifat autoregresif pada proses *caption generation*. Token yang diprediksi pada satu *timestep* akan digunakan sebagai input untuk *timestep* berikutnya. Jika pada satu tahap implementasi *scratch* menghasilkan token yang berbeda dari Keras, maka konteks untuk prediksi berikutnya juga berubah. Akumulasi perbedaan ini dapat menyebabkan *caption* akhir menyimpang dari hasil Keras dan menurunkan nilai evaluasi.

Meskipun demikian, implementasi *scratch* memiliki waktu eksekusi yang jauh lebih rendah. RNN *scratch* membutuhkan waktu **1.90 detik**, sedangkan RNN Keras membutuhkan **493.60 detik**. Pada LSTM, implementasi *scratch* membutuhkan waktu **3.66 detik**, sedangkan Keras membutuhkan **617.08 detik**. Hal ini menunjukkan bahwa implementasi *scratch* lebih ringan karena hanya menjalankan operasi inferensi yang diperlukan menggunakan NumPy. Namun, keunggulan waktu tersebut tidak diikuti oleh kualitas *caption* yang lebih baik. Dengan demikian, Keras lebih unggul dari sisi kualitas keluaran, sedangkan *scratch* lebih

sesuai digunakan untuk memahami dan memverifikasi mekanisme *forward propagation* secara manual.

Pengaruh Panjang Maksimum Caption terhadap BLEU-4

<i>Decoder</i>	<i>Implementasi</i>	BLEU-4	METEOR	<i>Time (detik)</i>
RNN	Keras	0.0573	0.3479	493.60
RNN	<i>Scratch</i>	0.0223	0.2966	1.90
LSTM	Keras	0.0557	0.3675	617.08
LSTM	<i>Scratch</i>	0.0214	0.3164	3.66

Nilai `max_caption_length` berfungsi sebagai batas maksimum panjang *caption* yang dapat dihasilkan model. Jika batas ini terlalu pendek, model berisiko berhenti sebelum menghasilkan kalimat yang lengkap. Sebaliknya, jika batas terlalu panjang, model memiliki peluang untuk menghasilkan token tambahan yang tidak diperlukan. Oleh karena itu, pemilihan nilai `max_caption_length` perlu mempertimbangkan panjang *caption* yang umum dihasilkan oleh model dan karakteristik data.

Pada hasil eksperimen, perubahan nilai `max_caption_length` dari 20 menjadi 30 dan 40 hampir tidak memengaruhi nilai BLEU-4 maupun METEOR pada kedua decoder. Pada LSTM, BLEU-4 tetap sebesar **0.0679** untuk seluruh variasi panjang maksimum. Pada RNN, BLEU-4 juga tetap berada pada nilai **0.0605**. Hasil ini menunjukkan bahwa sebagian besar *caption* telah selesai sebelum mencapai batas 20 token.

Kondisi tersebut terjadi karena proses *caption generation* berhenti ketika model menghasilkan token `<end>`. Dengan demikian, `max_caption_length` hanya berperan sebagai batas atas, bukan sebagai panjang keluaran yang wajib dicapai. Jika model sudah

menghasilkan token akhir sebelum mencapai batas maksimum, peningkatan nilai `max_caption_length` tidak akan mengubah *caption* yang dihasilkan. Hal ini menjelaskan mengapa peningkatan batas dari 20 ke 40 tidak memberikan dampak terhadap skor evaluasi.

Dari sisi efisiensi, `max_caption_length = 20` sudah memadai untuk eksperimen ini. Batas tersebut cukup untuk menampung *caption* yang dihasilkan model tanpa memotong kalimat, sekaligus menghindari proses iterasi yang tidak diperlukan. Oleh karena itu, peningkatan batas panjang *caption* di atas 20 tidak memberikan keuntungan yang berarti terhadap kualitas model.

E. Kesimpulan dan Saran

Berdasarkan eksperimen CNN, arsitektur terbaik diperoleh oleh model shared parameter shared_l2_f32-64_k3×3-3×3_max, yaitu model dengan 2 layer konvolusi, filter 32 dan 64, kernel 3×3-3×3, serta max pooling. Model ini memperoleh macro F1 validasi sebesar 0,7862 dan macro F1 test sebesar 0,7845. Perbandingan dengan model non-shared menunjukkan bahwa parameter sharing lebih efektif, karena model shared hanya memiliki sekitar 7,39 juta parameter tetapi menghasilkan performa lebih tinggi dibandingkan model non-shared yang memiliki sekitar 90,42 juta parameter dengan macro F1 validasi 0,6745. Selain itu, implementasi forward propagation CNN from scratch berhasil menghasilkan prediksi yang konsisten dengan Keras, dengan prediction agreement sebesar 1,0 dan selisih maksimum probabilitas sekitar 1,97e-06.

Pada bagian image captioning, model LSTM secara umum memberikan performa lebih baik dibandingkan Simple RNN. Model terbaik adalah lstm_preinject_layers2_hidden128 dengan BLEU-4 sebesar 0,0679 dan METEOR sebesar 0,3650, sedangkan model RNN terbaik adalah rnn_preinject_layers2_hidden128 dengan BLEU-4 sebesar 0,0605 dan METEOR sebesar 0,3592. Hasil ini menunjukkan bahwa mekanisme memori pada LSTM lebih sesuai untuk menangani data sekuensial caption. Namun, performa captioning secara keseluruhan masih relatif rendah, sehingga task image captioning terbukti lebih kompleks dibandingkan klasifikasi gambar karena model harus memahami informasi visual sekaligus membentuk urutan kata yang sesuai.

Untuk pengembangan selanjutnya, bagian CNN dapat ditingkatkan dengan menambahkan regularisasi, data augmentation, early stopping yang lebih sistematis, atau menggunakan pretrained CNN yang lebih kuat agar generalisasi model meningkat. Pada bagian image captioning, performa dapat ditingkatkan dengan encoder visual yang lebih baik, preprocessing caption yang lebih matang, beam search, dan attention mechanism agar decoder dapat memilih bagian gambar yang relevan saat menghasilkan kata. Selain itu, implementasi from scratch RNN/LSTM dapat divalidasi lebih rinci per timestep agar perbedaan dengan Keras lebih mudah ditelusuri dan hasil inference menjadi lebih konsisten.

F. Pembagian Tugas

Nama	Tugas
Muhammad Adam Mirza	Mengerjakan seluruh bagian image captioning RNN/LSTM, termasuk preprocessing, training, scratch implementation, evaluasi, analisis,
Devon Wiraditya T.	Mengerjakan seluruh bagian CNN, termasuk preprocessing, training, scratch implementation, evaluasi, analisis, serta membantu finalisasi repository dan laporan

G. Referensi

I. Goodfellow, Y. Bengio, dan A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

A. Zhang, Z. C. Lipton, M. Li, dan A. J. Smola, *Dive into Deep Learning*. 2023. Tersedia pada: <https://d2l.ai/>

F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY: Manning Publications, 2021.

Keras, "Keras Documentation." Tersedia pada: <https://keras.io/>

TensorFlow, "TensorFlow Documentation." Tersedia pada: <https://www.tensorflow.org/>

NumPy Developers, "NumPy Documentation." Tersedia pada: <https://numpy.org/doc/>

O. Vinyals, A. Toshev, S. Bengio, dan D. Erhan, "Show and Tell: A Neural Image Caption Generator," dalam *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

M. Tanti, A. Gatt, dan K. P. Camilleri, "Where to Put the Image in an Image Caption Generator," *Natural Language Engineering*, vol. 24, no. 3, pp. 467–489, 2018.

K. Papineni, S. Roukos, T. Ward, dan W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," dalam *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002.

Kaggle, "Flickr8k Dataset." Tersedia pada: <https://www.kaggle.com/datasets/adityajn105/flickr8k>

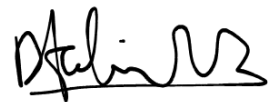
Kaggle, "Intel Image Classification Dataset." Tersedia pada: <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

H. Lampiran Form

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Iya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	-
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan: Pembuatan kode menggunakan tools seperti Gemini, dll	Ya	2



(18223015)



(18223039)